KLAi

SQL for analysts: ask a database the questions you ask pandas

How do we ask a database the same questions in the language every employer uses?

AI and Data Science Professional | Session 10 of 24 | Module C: Analyse and explain

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Write a query with SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY and LIMIT, and explain the order in which the database runs the clauses

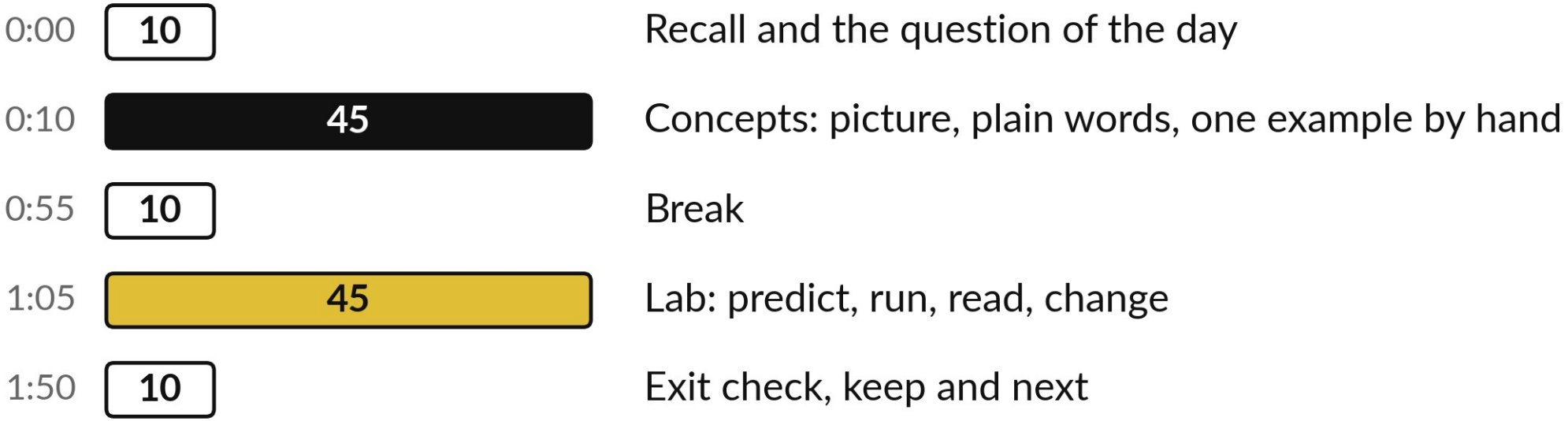


Write an INNER JOIN and a LEFT JOIN, say which records each keeps, and handle NULL correctly



Use CASE WHEN, a subquery and a CTE, and check every SQL answer against pandas

Your two hours: concepts first, then a SQL lab in Colab



Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. A merge ran without an error and revenue became 5.3 times too big. What happened, and what protects you?

2. Which join type keeps every order, including walk-ins with no customer_id?

3. What does pivot_table do that groupby alone does not?



Answer aloud first. Then we check together.

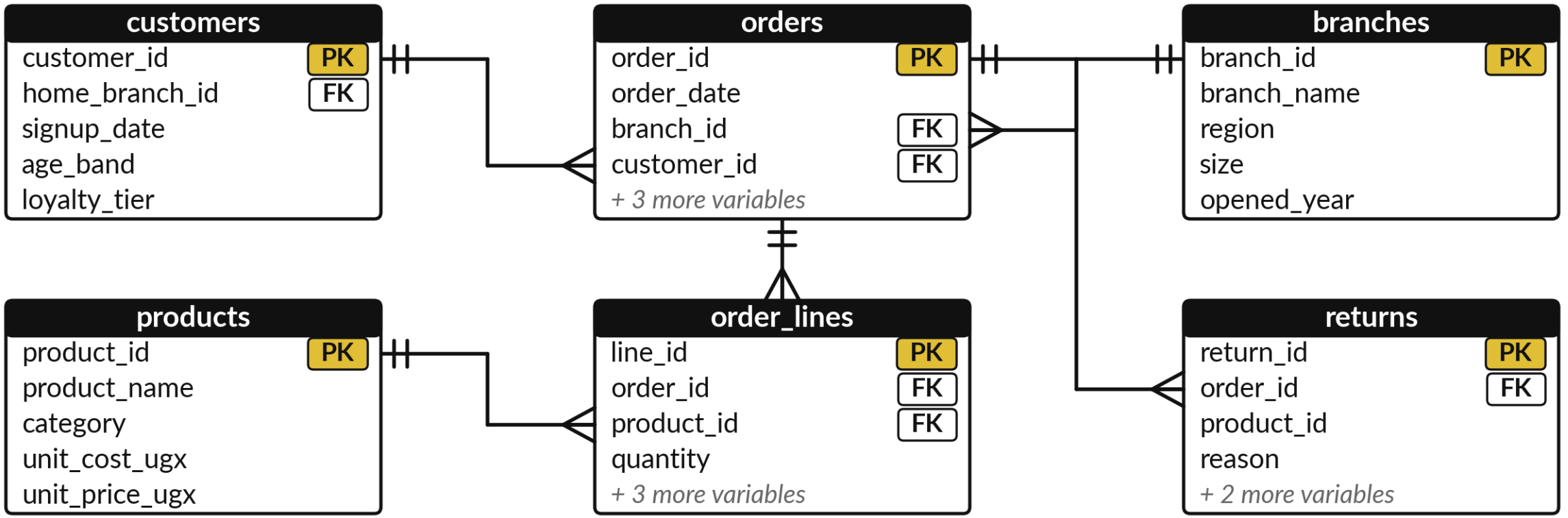
Answers to the three questions from Session 9

	Question	Short answer
	**A merge ran without an error and revenue became 5.3 times too big. What happened, and what protects you?**	Fan-out: a one-to-many merge repeated each order once per line. Check row counts before and after, and set validate.
	**Which join type keeps every order, including walk-ins with no customer_id?**	A left join from orders to customers. An inner join drops the records that have no match.
	**What does pivot_table do that groupby alone does not?**	It lays the grouped result out as a grid, with one variable down the side and one across the top, which is wide format.

Words we use today

	Word	What it means
	**Relational database (SQLite)**	Several tables linked by keys, stored in one file. SQLite is a free database that runs inside Python.
	**Query and clause**	A query is one question in SQL. A clause is one part of it, such as SELECT, WHERE or ORDER BY.
	**Aggregate function, GROUP BY and HAVING**	COUNT, SUM, AVG, MIN and MAX reduce records to one value. HAVING filters the groups that GROUP BY makes.
	**JOIN (INNER, LEFT)**	Combines two tables on a key. INNER keeps matches only. LEFT keeps every left record.
	**NULL (three-valued logic)**	A missing value. A test on NULL is neither true nor false but unknown, so use IS NULL.

A relational database holds several tables linked by keys: 9 tables in one SQLite file



Six of the 9 tables. PK: primary key. FK: foreign key. Three prongs: the many side.

The same question in Excel, in SQL and in pandas

	In Excel	In SQL	In pandas
	**Pick columns**	SELECT	df with a list of columns
	**Which sheet or table**	FROM	the DataFrame
	**AutoFilter on rows**	WHERE	a boolean filter
	**PivotTable**	GROUP BY with COUNT, SUM, AVG	groupby and agg
	**Filter the PivotTable totals**	HAVING	filter the grouped result
	**Sort**	ORDER BY, LIMIT	sort_values, head
	**VLOOKUP beside your full list**	JOIN, LEFT JOIN	merge

SQL keywords are written in capitals by convention. SQL ignores the difference.

SELECT, FROM, WHERE, ORDER BY and LIMIT: the three largest card orders at branch B06

```
SELECT order_id, order_value_ugx
FROM orders
WHERE branch_id = 'B06'
  AND payment_method = 'Card'
ORDER BY order_value_ugx DESC, order_id
LIMIT 3;
```

order_id	order_value_ugx
0023665	1165400
0022474	824600
0022280	452800

In plain words

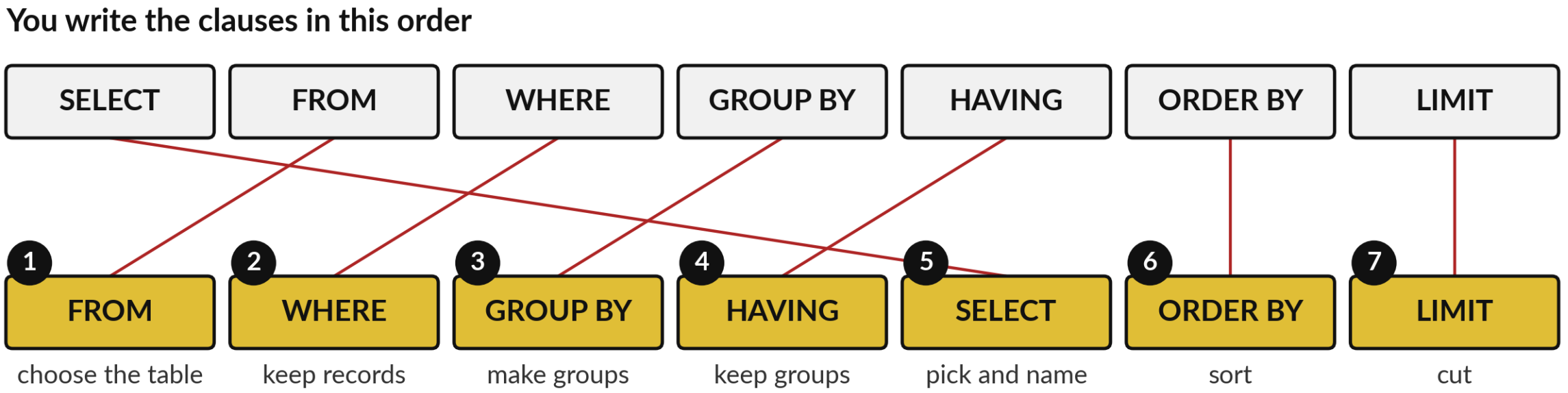
SELECT names the variables to return. FROM names the table.

WHERE keeps the records that pass a test. One equals sign tests equality, and text goes in single quotes.

ORDER BY sorts. DESC means largest first. A second variable breaks ties.

LIMIT 3 keeps three records. B06 has 257 card orders in all.

You write SELECT first, but the database runs FROM first and SELECT fifth



The database runs them in this order (gold boxes, numbered)

This order is why standard SQL forbids a SELECT alias in WHERE (SQLite allows it).

Aggregate functions reduce many records to one value

```
SELECT COUNT(*) AS records,
       COUNT(customer_id) AS filled,
       SUM(order_value_ugx) AS revenue,
       MAX(order_value_ugx) AS largest
FROM orders;
```

```
records filled    revenue largest
  30229  21058 1435614185 3104000
```

In plain words

COUNT(*) counts records: 30,229 orders.

COUNT(customer_id) counts filled values and skips NULL: 21,058.

The difference, 9,171, is the walk-in orders.

SUM adds up revenue: UGX 1,435,614,185. MAX is the largest order: UGX 3,104,000.

AS gives a result variable a name: an alias.

GROUP BY makes one result record per group, like a PivotTable

```
SELECT branch_id,
       COUNT(*) AS orders,
       SUM(order_value_ugx) AS revenue
FROM orders
GROUP BY branch_id
ORDER BY revenue DESC
LIMIT 4;
```

```
branch_id orders   revenue
      B01   5885 283331305
      B02   4830 238222000
      B04   3562 176738325
      B06   3852 170927325
```

In plain words

GROUP BY branch_id makes one group per branch.

Each aggregate function runs once per group.

ORDER BY can use the alias revenue, because SELECT has run by then.

B01 (Kampala Central) leads with UGX 283,331,305 from 5,885 orders.

HAVING filters the groups after GROUP BY has made them

```
SELECT reason, COUNT(*) AS n
FROM returns
GROUP BY reason
HAVING COUNT(*) > 450
ORDER BY n DESC;
```

```
    reason   n
   Damaged 683
Wrong item 473
   Expired 458
```

In plain words

returns has one record per returned line.

GROUP BY reason counts the returns for each reason.

HAVING COUNT(*) > 450 keeps only the groups with more than 450 returns.

The fourth reason, Changed mind, has 442 and is filtered out.

An aggregate function cannot appear in WHERE, because WHERE runs before the groups exist

```
SELECT reason, COUNT(*) AS n
FROM returns
WHERE COUNT(*) > 450
GROUP BY reason;
```

```
sqlite3.OperationalError:
misuse of aggregate: COUNT()
```

In plain words

This query asks WHERE to test a count.

WHERE runs second, before GROUP BY has made any group, so there is no count yet.

SQLite stops with an error.

Move the test to HAVING.

INNER JOIN keeps matches only, and LEFT JOIN keeps every record of the left table

Left table: orders

order_id	customer_id
O000001	C0495
O000002	NULL
O000004	C1815

Right table: customers

customer_id	loyalty_tier
C0001	Basic
C0495	Basic
C1815	Basic

Join key: customer_id

INNER JOIN — 2 rows

order_id	customer_id	loyalty_tier
O000001	C0495	Basic
O000004	C1815	Basic

Only keys found in both tables.

LEFT JOIN — 3 rows

order_id	customer_id	loyalty_tier
O000001	C0495	Basic
O000002	NULL	NULL
O000004	C1815	Basic

Every left record. No match gives NULL.

A real excerpt joined on customer_id. Gold cells are NULL (missing).

INNER JOIN with aliases: revenue by region

```
SELECT b.region, COUNT(*) AS orders,
       SUM(o.order_value_ugx) AS revenue
FROM orders AS o
INNER JOIN branches AS b
        ON o.branch_id = b.branch_id
GROUP BY b.region;
```

```
  region orders   revenue
 Central  14060 679043495
 Eastern   6599 319196035
Northern   2929 140789695
 Western   6641 296584960
```

In plain words

FROM orders AS o gives the table a short name, an alias.

ON says which variables must match: the join key.

Each branch appears once in branches, so the join is many-to-one and keeps every order.

Central leads: UGX 679,043,495 from 14,060 orders.

LEFT JOIN keeps the walk-in orders, and their loyalty tier is NULL

```
SELECT c.loyalty_tier,
       COUNT(*) AS orders
FROM orders AS o
LEFT JOIN customers AS c
       ON o.customer_id = c.customer_id
GROUP BY c.loyalty_tier
ORDER BY orders DESC;
```

```
loyalty_tier orders
       Basic  13075
        NULL   9171
      Silver   5801
        Gold   2182
```

In plain words

Every order is kept, so the counts add up to all orders.

The NULL group holds the 9,171 walk-in orders: no customer, so no match.

An INNER JOIN to customers would return 21,058 rows, which is 9,171 fewer than the 30,229 orders.

GROUP BY puts all NULL values into one group.

A test on NULL is neither true nor false, so WHERE drops the record

Test written in WHERE: customer_id <> 'C0495'

order_id	customer_id	test gives	WHERE keeps the row?
O000001	C0495	FALSE	no
O000002	NULL	UNKNOWN	no
O000004	C1815	TRUE	yes

A missing value (NULL) means unknown. Is an unknown value different from C0495? SQL answers UNKNOWN, and WHERE keeps only TRUE.

21,050	8	9,171
TRUE: kept	FALSE: dropped	UNKNOWN: dropped

Three-valued logic: a test can be TRUE, FALSE or UNKNOWN.

NULL means unknown, so any comparison with it is UNKNOWN.

WHERE keeps only TRUE.

To find NULL, write IS NULL. To keep it in a test, add OR customer_id IS NULL.

IS NULL finds the missing values that an equals test cannot see

```
SELECT SUM(customer_id <> 'C0495') AS test_true,
       SUM(customer_id =  'C0495') AS test_false,
       SUM(customer_id IS NULL)    AS is_null
FROM orders;
```

```
test_true test_false is_null
    21050          8    9171
```

In plain words

SUM(customer_id <> 'C0495') adds up the TRUE results: 1 for TRUE, nothing for UNKNOWN.

SUM(customer_id = 'C0495') adds up the orders of that customer.

SUM(customer_id IS NULL) counts the NULL values.

The three add up to 30,229, every order.

CASE WHEN makes a new categorical variable inside a query

```
SELECT CASE
         WHEN order_value_ugx < 20000 THEN 'Small'
         WHEN order_value_ugx < 100000 THEN 'Medium'
         ELSE 'Large'
       END AS band,
       COUNT(*) AS orders
FROM orders
GROUP BY band
ORDER BY orders DESC;
```

```
  band orders
Medium  14486
 Small  13529
 Large   2214
```

In plain words

CASE tests the conditions from top to bottom and returns the first match.

ELSE catches everything left.

It is the IF function of SQL. The pandas twin is pd.cut with right=False.

The bands are in UGX: Small under 20,000, Medium under 100,000, Large the rest.

SQLite lets GROUP BY use the alias band. Other databases need the CASE expression repeated.

A subquery is a query inside brackets that supplies a value to another query

```
SELECT COUNT(*) AS above_average
FROM products
WHERE unit_price_ugx >
      (SELECT AVG(unit_price_ugx) FROM products);
```

```
above_average
            20
```

In plain words

The inner query computes the average unit price: UGX 11,523.

The outer query counts the products priced above it.

20 of 150 products are above average.

The inner query runs first.

A CTE (WITH) names a subquery first, so the main query reads in order

```
WITH by_branch AS (
  SELECT branch_id, SUM(order_value_ugx) AS revenue
  FROM orders
  GROUP BY branch_id
)
SELECT COUNT(*) AS branches,
       MAX(revenue) AS best
FROM by_branch;
```

```
branches      best
       8 283331305
```

In plain words

WITH names a temporary result: a common table expression (CTE).

by_branch has one record per branch.

The main query then reads it like a table.

It finds 8 branches and a best revenue of UGX 283,331,305.

sqlite3 and pandas.read_sql: run SQL in Colab and compare with pandas

```
# orders is loaded from orders.csv
import sqlite3
con = sqlite3.connect('lakeside.db')
sql = "SELECT COUNT(*) AS n FROM orders"
sql += " WHERE branch_id = 'B04'"
a = pd.read_sql(sql, con).n[0]
b = (orders.branch_id == 'B04').sum()
print(a, b, a == b)
```

```
3562 3562 True
```

In plain words

sqlite3.connect opens the database file.

read_sql runs a SQL query and returns a DataFrame.

The pandas filter gives the same answer: 3,562 orders at branch B04.

a == b prints True when SQL and pandas agree.

SQL and pandas side by side: the same answer, written two ways

	Question	SQL	pandas
	**Revenue by branch**	GROUP BY branch_id with SUM	groupby then sum
	**Groups with more than 450 returns**	HAVING COUNT(*) > 450	value_counts, then a filter
	**Add the region to each order**	INNER JOIN or LEFT JOIN … ON	merge on the key
	**Order value in bands**	CASE WHEN … END	pd.cut(..., right=False)
	**Walk-in orders**	WHERE customer_id IS NULL	isna().sum()
	**Products above the average price**	a subquery with AVG	a comparison with mean()

Your employer's database may not be SQLite: four places where the dialect differs

Task	SQLite (the lab)	Other databases (check the documentation)
Month of a date	strftime('%Y-%m', order_date)	PostgreSQL: DATE_TRUNC or TO_CHAR. SQL Server: FORMAT or DATEFROMPARTS. MySQL: DATE_FORMAT.
Keep the first rows	LIMIT 3	SQL Server and Access: SELECT TOP 3. Oracle: FETCH FIRST 3 ROWS ONLY.
A SELECT alias in WHERE	Accepted	The SQL standard and many databases refuse it. Repeat the expression or use a CTE.
RIGHT and FULL OUTER JOIN	Only from version 3.39	Available in PostgreSQL, SQL Server, Oracle and MySQL (full outer join in MySQL needs a workaround).

The ideas carry over. The spelling of a few functions changes, so keep the documentation open at work.

Where you will use this at work: ask the company database directly

	Your job	The tables you meet	What today lets you do with your own tables
	**Credit analyst at a bank in Kampala, using SQL**	loans: one record per loan. customers: one per borrower.	GROUP BY branch and month for arrears. LEFT JOIN to keep borrowers with no loan.
	**Revenue analyst at a mobile money or telecom company**	Transactions, agents and regions.	Join, group and rank agents with a CTE, then export the result to Excel or Power BI.
	**Health records officer at a district office**	Clinic reports: one record per district per month.	COUNT the blank months with IS NULL before totalling a district.
	**Data analyst at a retailer or an NGO**	Orders, products, branches or beneficiaries.	Answer a manager question in minutes, and prove it with a pandas check.

Read the recipe before the lab: one query with every clause

```
SELECT b.region,
       SUM(o.order_value_ugx) AS revenue
FROM orders AS o
LEFT JOIN branches AS b
       ON o.branch_id = b.branch_id
WHERE o.promo_week = 0
GROUP BY b.region
HAVING SUM(o.order_value_ugx) > 1e8
ORDER BY revenue DESC
LIMIT 3;
```

In plain words

SELECT and FROM name what to return and where from.

LEFT JOIN ... ON adds the region and keeps every order.

WHERE filters records before grouping.

GROUP BY and HAVING make groups and filter them.

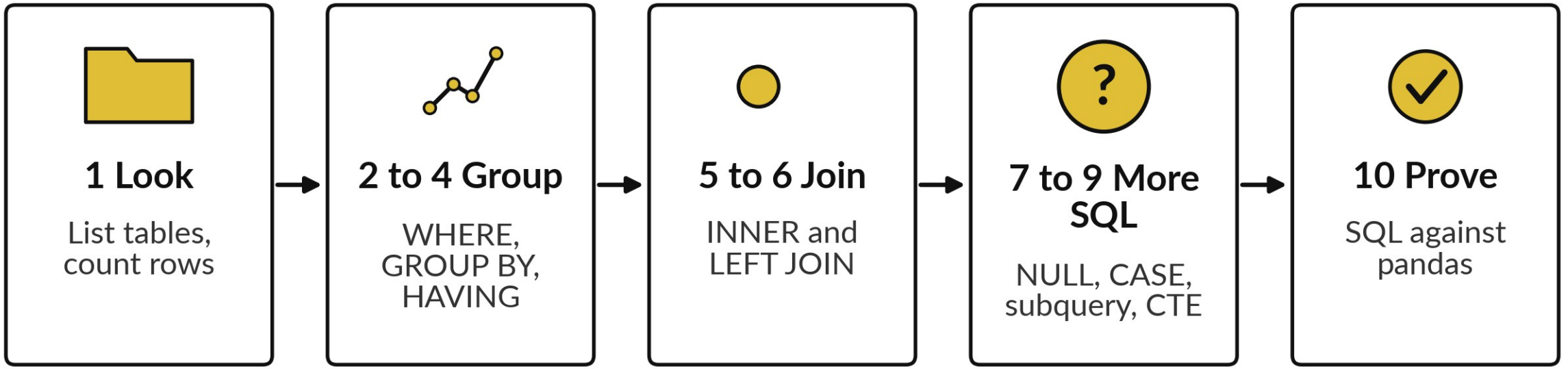
ORDER BY and LIMIT sort and cut.



Part 2: the SQL lab

Close the slides. Open the notebook.

The lab has ten steps and ends with a proof table you keep



Ten steps in 45 minutes. For each: predict, run, read, change. Four answers are rebuilt in pandas, and your proof table (a reconciliation of the SQL answer against the pandas answer) records the match.

Check your numbers before you move on

What you compute	You should get
Tables in the database, and records in orders	9 and 30,229
Card orders at branch B06	257
Top branch by revenue in SQL	B01, UGX 283,331,305
Return reasons with more than 450 records	Damaged 683, Wrong item 473, Expired 458
Inner join to customers: rows, against the left join	21,058 against 30,229
Walk-in orders (IS NULL)	9,171
Medium, Small and Large orders	14,486, 13,529 and 2,214
Products above the average price	20 of 150

If your number differs, find the task that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
no such column: branch_name	The variable is in another table, or misspelt.	Check the table with SELECT * FROM table LIMIT 3.
misuse of aggregate: COUNT()	An aggregate is in WHERE.	Move the test to HAVING.
A WHERE test returns fewer records than expected	NULL values make the test UNKNOWN.	Count IS NULL first, and add OR variable IS NULL if needed.
Revenue much bigger after a JOIN	The right table repeats the key: fan-out.	Count the rows before and after. Summarise the right table first.
The result order changes between runs	No ORDER BY, so the order is not defined.	Add ORDER BY, with a second variable to break ties.

Exit check: three questions, then compare with the short answers

Question	Short answer
Which clause runs first in a SQL query, and which runs fifth?	FROM runs first. SELECT runs fifth, after WHERE, GROUP BY and HAVING. That is why the SQL standard does not let WHERE use an alias made in SELECT. SQLite allows it, many databases do not.
An INNER JOIN of orders to customers returns 21,058 rows and a LEFT JOIN returns 30,229. Why?	9,171 orders have a NULL customer_id, so they have no match. The inner join drops them and the left join keeps them.
Why does WHERE customer_id <> 'C0495' not return the walk-in orders?	A comparison with NULL is UNKNOWN, and WHERE keeps only TRUE. Add OR customer_id IS NULL.

Save your query book with its proof table

What you keep from today

Keep your Colab query book: ten SQL queries on the Lakeside database, each with a proof table that shows the SQL answer next to the pandas answer.

Our question today

How do we ask a database the same questions in the language every employer uses?

Next session

Session 11: Exploring and visualising with Python. We draw the numbers so that a reader sees the finding.

Take away

Your lab file

A query book

The habit

SQL then pandas

Next session

Charts